

Request a web page

Runs  
Server side  
Application

Returns a HTML doc

Client

\* Server side web application

Request a web page

Runs

Returns

Client side

Application

a HTML doc

Server

\* Client side web application

\* .NET framework

The languages use to support .NET

1) C# Sharp, C++, C, F# Sharp, VB (Visual Basic)

2) CLR (Common Language Runtime)

It is a compiler language.

3) CLS (Common Language Specification)

4) CTS (Common Type System)

1) One of the most essential component of .NET framework is CLR. CLR is the environment where all the programs of .NET framework is executed.



- 2) CLR allows the Execution of code across different platforms by transmitting code into IL.
- 3) IL is converted into Machine Language during Execution by JIT (Just In Time Compiler)
- 4) During JIT compilation the code is also checked for Type Safety.
- 5) Type Safety ensures that objects are always accessed in a Computable way if value of 8 byte is given to a 4 byte when severe error will occur by CLR & error will attempt to trap
- 6) CLR consist of common rules that must be followed by all languages using .NET framework. These set of rules are called as CLS
- 7) The CLS enables an object or application to interact with the objects or application of another language.
- 8) The classes it follows the rules specify by CLS are termed as CLS compliant classes.
- 9) One of the Specification define in CLS is Common Type System (CTS).
- 10) CTS provide a Type System that is common across all languages.
- 11) CTS is defines rules that ensure that the data object written in various languages are to interact to each other.
- 12) During Execution CLR performs the following steps:
  - a) Loading Assemblies & Identifies Namespaces
    - i) Assemblies are loaded into Memory for the CLR to identify Namespace for the code in assembly.
    - (ii) Namespace are collection of classes (Similar to package in Java). Namespaces are used to Organise classes in hierarchy.



(iii) The Namespace have public access & can be Modified.

b) JIT Compilation:-

(i) Before Execution the IL (Intermediate language) is converted machine language using JIT Compiler.

(ii) The IL code is examined for the following points

(iii) The memory location that code needs to access are available.

(iv) Methods are called only through properly defined types.

(v) IL has been correctly generated

c) Garbage Collection :-

(i) The Garbage Collection are processed begins after the JIT compilation.

(ii) It Manages the allocation & de-allocation of the Memory used by an application.

(iii) When you create an object the CLR allocates Memory for the object from the Managed heap.

\* .NET framework library

System, Collection, CLR, CTS, etc.

\* ASP.NET

\* Visual Studio.

## C# language

### 1) 'HELLO WORLD' Program

using System;  
class Test

```
{
    public static void main (String optional args[])
    {
```

```
        Console.WriteLine("Hello world"); (Console.WriteLine)
        Console.ReadKey(); (Read value from user optional)
    }
}
```

### \* Keywords

- 1) Keywords are the reserved words
- 2) There are specific type of keywords used in C#
- 3) The list of keywords

### \* Identifiers

Identifiers are the names given to classes, variables, etc.

### \* What are Variables?

Variables are the names given to memory location.



## \* Data Types

Create a Table of Data Types.

| C#      | VB      | NET      | Size (Byte) |
|---------|---------|----------|-------------|
| byte    | byte    | byte     | 1           |
| Short   | Short   | Int 16   | 2           |
| Int     | Integer | Int 32   | 4           |
| long    | Long    | Int 64   | 8           |
| float   | Single  | Single   | 4           |
| double  | double  | double   | 8           |
| decimal | decimal | decimal  | 16          |
| Char    | Char    | Char     | 2           |
| String  | String  | String   |             |
| bool    | Boolean | Boolean  | 1 bit       |
|         | Date    | DateTime |             |

\* WAP to accept number from the user.

```

using System;
class Factorial
{
    public static void main()
    {
        int number, f=1;
        Console.WriteLine("Enter a number");
        String s = Console.ReadLine();
        num = Convert.ToInt32(s);
        for (int i=1; i<=num; i++)
    }

```



```
f = f * i;
```

```
}
```

```
Console.WriteLine("factorial of {0} is {0}", num, f);
```

```
}
```

\* WAP to accept a number through command line & calculate fibonacci series till number

```
using System;
```

```
class fibonacci
```

```
{
```

```
public static void main (String args [])
```

```
{
```

```
int x, a=0, b=1, c=0;
```

```
x = Convert.ToInt32(args[0]);
```

```
Console.WriteLine("a=" + a + "\n b=" + b);
```

```
int i=2;
```

```
while(i <= x)
```

```
{
```

```
c = a + b;
```

```
Console.WriteLine(c);
```

```
a = b;
```

```
b = c;
```

```
i++;
```

```
}
```

```
Console.ReadKey();
```

```
}
```

```
}
```

\* foreach loop

```
foreach (type Variable in Expression)
{
    Body of the loop
}
```

```
int arr[] = { 1, 2, 3, 4, 5 };
foreach (int x in arr)
{
    Console.WriteLine(x);
}
```

\* WAP to accept five number from the user & find largest of the five.

```
using System;
class Largest
{
    public static void main()
    {
        int[] arr = new int[5];
        Console.WriteLine("Enter 5 Numbers");
        for (int i = 0; i <= 5; i++)
        {
            arr[i] = Convert.ToInt32(
                Console.ReadLine());
        }
        int max = arr[0];
        for (int x in arr)
        {
            if (x >= max)
            {
                max = x;
            }
        }
        Console.WriteLine("Largest Number is: " + max);
    }
}
```



```
max = x;
```

```
console.WriteLine(max);
```

```
Console.ReadKey();
```

```
}}
```

## Type Commission

1) Convert Total 32 (x);

```
int 32.Parse();
```

2) float - Convert To Single (x)

```
float.Parse(x);
```

```
double.Parse(x);
```

## Switch Statement

using System;

```
class vowel
```

```
{
```

```
public static void main()
```

```
{
```

```
Console.WriteLine("Enter a character");
```

```
char c = char.Parse(Console.ReadLine());
```

```
switch (c)
```

```
{
```

```
case 'a': Console.WriteLine("It is a vowel");
```

```
break;
```

```
case 'e': Console.WriteLine("It is a vowel");
```

```
break;
```

```

case 'i' : Console.WriteLine("It is a vowel");
break;
case 'o' : Console.WriteLine("It is a vowel");
break;
case 'u' : Console.WriteLine("It is a vowel");
break;
default : Console.WriteLine("It is a consonant");
break;
}
}

```

\* WAP to find volume of a cylinder, sphere, cube.

using System;

class Volume

{  
public void cylinder(float radius, float height)

{  
float volume = 3.14 \* radius \* radius \* height;

Console.WriteLine("Volume of cylinder is " + volume);  
}

public void sphere(float radius, float height)

{  
float volume =  $\left(\frac{4}{3}\right) * 3.14 * radius * radius * radius$ ;

Console.WriteLine("Volume of sphere is " + volume);  
}

public void cube(int radius)

{  
int volume = radius \* radius \* radius;

Console.WriteLine("Volume of cube is " + volume);  
}



```

class Mainclass
{
    public static void main ()
    {
        Volume v = new Volume ();
        v.Cylinder (2,3);
        v.Sphere (1,0);
        v.Cube (2);
    }
}

```

Ans

```

class Mainclass
{
    public static void main ()
    {
        float rad, hei;
        console.WriteLine ("Enter radius & herght");
        rad = Convert.ToSingle (console.ReadLine());
        hei = float.Parse (console.ReadLine());
        Volume v = new Volume ();
        v.Cylinder (rad, hei);
        v.Sphere (rad, hei);
        v.Cube (rad);
    }
}

```

\* Default Values

All Integer

-

0

|         |   |        |
|---------|---|--------|
| char    | - | '\x00' |
| float   | - | 0.0f   |
| double  | - | 0.0d   |
| decimal | - | 0.0m   |
| bool    | - | false  |
| enum    | - |        |

All reference types are null.

fixed length

Value Types (stack)

Reference Type (heap)

User-Defined

Pure-Defined

- Classes
- Interface
- Arrays
- Delegates

- Numeric
- Boolean
- Character Types
- String
- Object

\* Boxing & unboxing

Value to Reference :- Boxing

Reference to Value :- unboxing

\* Operators & Expression.



# \* Arithmetic

+ Add

- Sub

\* mul

% Modulus

/ Divide

Integer Arithmetic

Real Arithmetic :- Decimal

Mixed-mode Arithmetic :- Decimal + Real

# \* Relational

<= less than =

>= greater than =

< less than

> greater than

= Equal to

!= Not equal to

# \* Logical

&& logical AND

|| logical OR

! logical NOT

& Bitwise AND

| Bitwise OR

^ Bitwise logical XOR

<< Shift left

>> Shift right

~ Ones complement

# \* Conditional

? : (ternary)

# \* Increment/Decrement

++

--

a++ post increment

++a pre

a-- post decrement

--a pre

# \* Assignment

=

+=

-=

/=

%=

# Special operation

is (Relational)

as (Relational)

typeof (type operation)

sizeof (size of)

new (object creator)

def (member are)

checked

unchecked

checking

for prevention

## \* pass by value & pass by reference

By default method parameters are passed by value. The parameter are declared with no modifier is pass by value & is called value parameter.

When a method is invoked the values of actual parameters are assigned to the formal parameter.

The value of the actual parameter that is passed is not changed by any changes made in the corresponding formal parameter because the methods refer to the copy of those variables.

Code :-

Using System;

class passByValue

{

public static void main()

{

int x=10;

change(x);

console.WriteLine("x=" + x);

static void change (int x)

{

a = a+10;

}

## \* pass By Reference

Unlike a value parameter a reference parameter does not create a new storage location as the



instant it represents the same storage allocation as the actual parameter in the method invocation.

Using System;

Class pass By Reference

```
public static void main ()
```

```
{
    int m = 10;
    modify (m); // This will not work
    modify (ref m);
}
```

```
static void modify (ref int a)
```

```
{
    a + 10;
}
```

\* When a formal parameter is declared as ref the corresponding argument in the method invocation must also be declared as ref.

\* The output parameter

using System;

class out parameter

```
{
    static void square (int x, out int y)
```

```
{
    y = x * x;
}
```

Page No. :  
Date : / /

```
public static void main ()
{
```

```
    int sax;
    Square (10, sax);
    Console.WriteLine (sax);
}
```

## \* Delegate

Syntax :

[Modifier] delegate (return-type) (delegate-name params)

- 1) It act on a function pointer that allows the Methods of the references.
- 2) delegate is a type-safe Method function.
- 3) It can reference both static & instances method.
- 4) They are the foundation of events & as function in C#.
- 5) why we use delegate?
  - 1) for implementing callback
  - 2) for handling events
  - 3) for writing flexible, reusable code where behaviour can be passed as parameter.



## \* Name Space

- There are two main types of C#

- 1) Your Applications Namespace
- 2) System Namespace : System Namespace is a part of .Net library.

```

class
namespace MyCompany.MyApp
{

```

```

    public class product
    {

```

```

        }
    }
}
import mycompany.mypapp;

```

```

    product

```

## \* Assemblies

1) Assemblies are the physical file that contain the compiled code.

2) Assemblies file have the extension (.exe).

3) If they are Standalone applications & .dll if they are reusable components.

4) A ~~strict~~ Relationship does not Except Namespace



Assemblies can contain Multiple Namespace more than one assemblies file can contain classes in the same Namespace. Namespace are a logical way to group classes.

5) Assemblies are a physical package to distribute

## \* Asp. net Controls

using the page class :- Every page is a custom class that inherit from System.Web.UI.Page. By inheriting in this class you will web page acquires a number of properties & ~~new~~ methods.

## \* Properties (Basic page)

- 1) isPostBack
- 2) EnableViewState
- 3) Application
- 4) Session
- 5) Cache
- 6) Reentrant
- 7) Response
- 8) Server
- 9) User

## \* Sending the user to New page.

```
<a href="http://www.google.com">Google </a>  
Response.Redirect("dashboard.aspx");  
Response.Transfer("filename");
```

\* Using application events the global.asax files this files allows us to write code that response to global application events. These events are fired at various points during lifetime of web application. Including when the application domain is first created.



\* choose website add new item / Select global application class file then click OK.

\* The global.aspx file look similar to a normal .aspx file except that it cannot contain any html tags.

```
<%@ Application language = "(#" %>
<Script language = "(#" run at = "Server">
protected void Application_EndRequest(object Sender,
EventArgs e)
```

```
{
    Response.Write("<hr/> This page was Served at"
+ DateTime.Now.ToString());
}
</script>
```

\* Additional Application Events

Application\_Start()

Application\_End()

Application\_BeginRequest()

Application\_EndRequest()

Session\_Start()

Session\_End()

Application\_Error()

## Asp.Net web forms

\* Click on Visual Studio 2022 / New Project

Step 1: Asp.Net web forms app framework template

Step 2: Sample web application

Step 3: Solution Explorer

Step 4: Content.aspx → right click

Step 5: Default.aspx

Step 6: Run

### \* Default.aspx \*

```
<html> <head> <body>
<form id="form1" runat="Server">
<input type="text" id="USD" runat="Server">
<h1> Convert USD to Euro </h1>
<b> <label id="result" runat="Server"> </label>
</b>
<input type="submit" value="Submit" onclick="
Convert" runat="Server">
</form> </body> </head> </html>
```

### \* Default.aspx.cs \*

```
protected void Convert_Submit (object
Sender, EventArgs e)
{
```

```
decimal USD = Decimal.parse (usp.value)
decimal Euro = usp * 0.85 m;
result.InnerText = USD to String () + " Euro";
```



## \* AutoPostBack \*

→ The default value for AutoPostBack is false.

[Page object is created from .aspx file]

[page.init event occurs]

[Controls are repopulated with information from view State]

[page.load events occurs]

[All other event occurs (click, change)]

[page.preRender event occurs]

[Control information is stored in view State]

[HTML form page is rendered]

[page.unload event occurs]

[Page object is released from Memory]

fig - The postBack processing sequence.

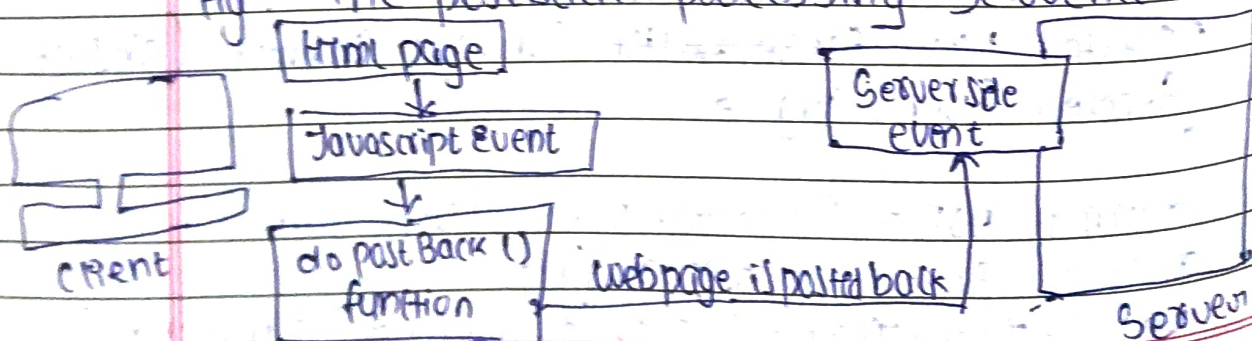


fig - An automatic postBack

## \* Validation Controls \*

- 1) Required field Validator.
- 2) Range Validator.
- 3) Compare Validator.
- 4) Regular Expression Validator.
- 5) Custom Validator.

## \* Server Side Validation \*

A page can automatically view Validator when the page is Submitted or Manually define in Your code. For Automatic Validation Every button has a `Causes Validation property`.

## Client Side Validation

When a user place a positive <sup>passes</sup> the same error message (Server). will appear without the page loading to be Submitted & returned from the Server. This Increases the responsiveness for web page. However even if the webpage validate successfully on the Client Side Asp.net still revalidates it when its receive at the Server.

## Manual Validation

\* Three base to achieve Manual Validation

- 1) use Your own code to Verify values. In This case Your use any Asp.net Validation controls.
- 2) Disable Enable ClientScript property for each validation control.
- 3) This allows an Invalid page to be Submitted.
- \* Add a button with `pagesValidation` set to false. when the button



is click manually page \* is validate by clicking  
page.validateMethod. Then Exam Member is validate  
& decide what to do.

## \* Validation with regular Expression

Character

\* Zero or more

+ One or more

{ }

Some Value

|

Single pipe Sign

[ ]

Match the value in specific range

[^ ]

[^A-c] D, E, f will validate

Some Character

\s

Except space

\S

\d {Digit}

Any Digit [0-9]

\D

Any character is in Digit

\w

{word}

\W

Example:-

Email address \S+@\S+\.\S+

Password \w+

Specific-length password \w{4,10}

Advanced password [a-zA-z]\w{3,9}

Limited-length field \S{4,10}

### Chp - Rich Controls

Calendar

```
<asp:Calendar  
runat="Server"
```

```
id="MyCalendar">
```

Ad Rotator

```
<asp:AdRotator
```

```
id="Ads"
```

```
runat="Server"
```

```
AdvertisementFile="Advanced"
```

```
Target="black"  
KeywordFilter="Computers"  
>
```

#### \* Calendar \*

The Calendar controls provide a ~~miniature~~ <sup>miniature</sup> calendar that you can place in any webpage. It is one of the Rich Control. It presents a complete <sup>Month</sup> One Month view. The user can navigate from Month to Month.

#### \* Calendar Selection Mode \*

Date, Day, Week, Month, Year. First Day of week property allows us to configure how the week is shown. You can also allow Multiple Selection.

#### \* Formatting the Calendar & Properties \*



Calendar Style  
Day Header Style

Calendar Day  
Date

Day Style

Next prev Style

Other Month Day Style

Selected Day Style

Selector Style

Title Style

Today Day Style

Weekend Day Style

Is weekend

Is today

Is Other Month

Is Selectable

## Advertisement file

< Advertisements >

< Ad >

< ImageURL > infotech.jpg < /ImageURL >

< NavigateURL > http://www.infotech.com < /NavigateURL >

< Impressions > 1 < /Impressions >

< Keyword > Computer < /Keyword >

< /Ad >

< Ad >

< /Ad >

< /Advertisements >

## \* Target Values \*

### \* The four land parent values

1) blank

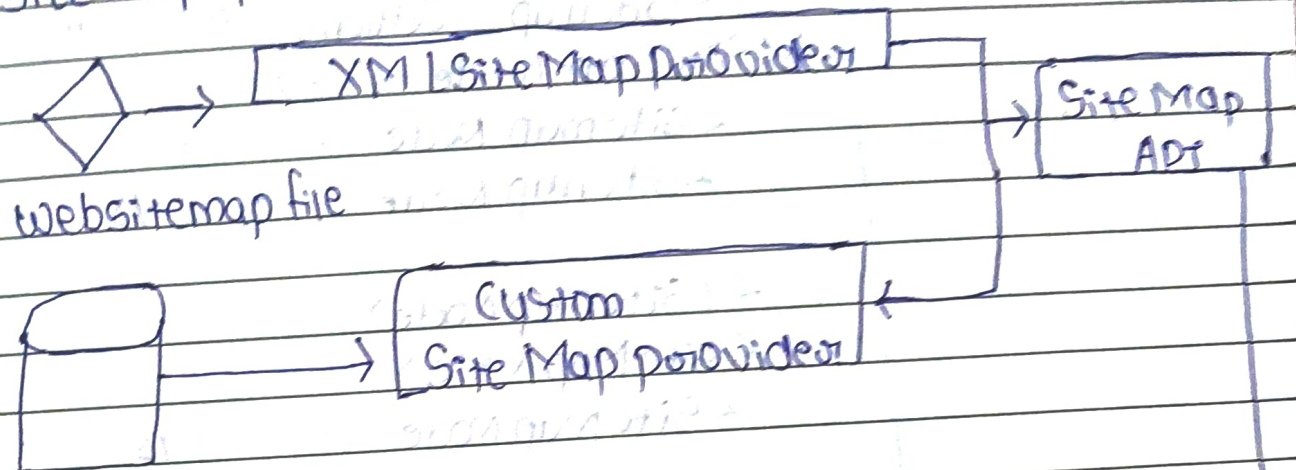
2) parent

3) Self

4) Top

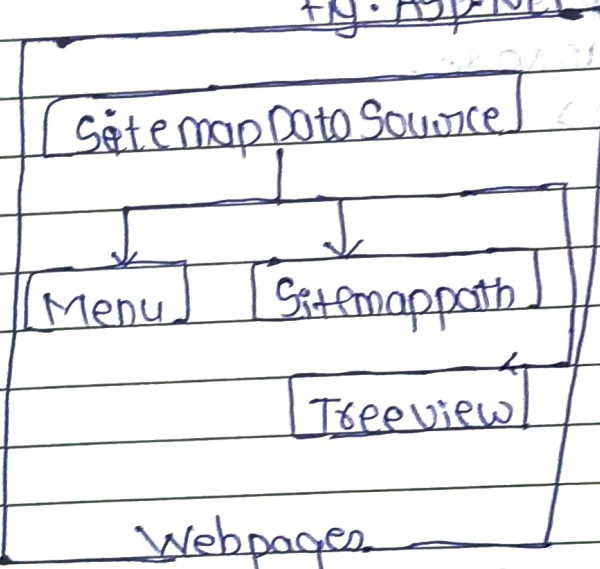
# SITE MAP PATH

Site map based Navigation is the Starting point in Site map provider.



Custom Database

fig: ASP.NET Navigates with Sitemaps



\* Rules \*

Rule 1: Begins with <Sitemap> Elements

Rule 2: Each page is represented with a <SitemapNode> Element

Rule 3: <SitemapNode> Element can contain other <SitemapNode> Element

Rule 4: Every Sitemap begins with a single <SitemapNode>

Rule 5: Duplicate URLs are not allowed

Teacher's Signature:.....



til here

## Steps To Create a Sitemap

Website → New item → Site map  
By Default the Sitemap consist of Basic Structure  
With tree Sitemap

